

Fatigue-State-Aware, Uncertainty-Robust Scheduling with Memory-Review Coupling for Cognitive-Load-Aware Task Planning

Vaishvi Khandelwal Prathamesh Sankhe Amal Verma Vedaang Devendra Pagnis

Department of Computer Engineering and Information Technology
Veermata Jijabai Technological Institute

Mentor: Prof. S. Shrawne

{vkhandelwal, psankhe, averma, vpagnis}@vjti.ac.in

Abstract

Axiom’s current task scheduler places tasks using hand-tuned greedy heuristics without an explicit objective function, fatigue accumulation dynamics, or stochastic uncertainty bounds. We formulate cognitive-load-aware scheduling as an optimization problem over a dynamic capacity state comprising a circadian ceiling (derived from user profiles) and an accumulating/recovering fatigue variable. We propose a deterministic mixed-integer program (**P-DET**) implementable in CP-SAT, a sample-average chance-constrained extension (**P-CC**) consuming predictive distributions from the LLM estimator (M4) and adaptive profile (M3), and a retention-gain coupling mechanism linked to a Bayesian memory retention model. We evaluate exact, heuristic (simulated annealing), and rolling-horizon solvers against fixed-interval and baseline Axiom schedulers using simulated learners whose data generator structurally differs from the planner’s model. This work situates the formulation against recent (2020–2026) literature on fatigue-aware, robust, conformal, and LLM-assisted scheduling. All evaluation claims remain relative to the formal model and simulated benchmarks.

Keywords: cognitive load, task scheduling, fatigue management, mixed-integer programming, CP-SAT, chance-constrained programming, robust optimization, spaced repetition.

1 Introduction

Personal task planning systems frequently claim “cognitive-load awareness”, yet scheduling logic in practical platforms (e.g., Axiom [1]) relies on greedy heuristics multiplying deadline proximity, static priority, and time-of-day energy levels. Lacking a global objective, such systems cannot formally guarantee schedule optimality or workload bounds.

Our architecture organizes task planning into a four-module loop: Module M1 provides the load function $L(\delta, p, \kappa)$ grounded in Cognitive Load Theory [2]; Module M4 generates per-task estimates $(\mu_p, \sigma_p, \mu_\delta, \sigma_\delta)$ using LLM estimators and user profiles; Module M3 maintains the adaptive user profile and Bayesian retention model [3]; and Module M2 (this paper) executes optimal scheduling. The loop operates sequentially as $M4 \rightarrow M2 \rightarrow M3 \rightarrow M4$. Whereas static models enforce rigid window budgets $\sum_{i \in \mathcal{S}_t} \ell_i \leq L_{\max}(u, t)$ [3], Module M2 introduces a time-resolved capacity state tracking fatigue accumulation and recovery.

Research Questions. **RQ1:** Does fatigue-aware capacity optimization reduce cognitive overload and deadline violations compared to greedy and fixed-interval baselines? **RQ2:** Does propagating M3/M4 uncertainty into chance constraints reduce realized overload, and at what cost to task completion rate? **RQ3:** How closely does a metaheuristic solver approximate CP-SAT exact solutions across realistic problem sizes? **RQ4:** How sensitive are schedule outputs to load function formulations and unobservable fatigue model hyper-parameters?

Contributions.

1. A linear capacity-state model merging depleting workload budgets with circadian energy curves (**C1**).
2. A MIP formulation (**P-DET**) enforcing hard deadlines, precedence, atomic tasks, and switching penalties, proven tight under linear continuous slacks (**C2**).
3. A sample-average chance-constrained formulation (**P-CC**) integrated with predictive distributions from LLMs and marginal retention gains (**C3**).
4. An evaluation protocol employing misspecified learner simulations and real study traces (**C4**).
5. A systematic positioning against recent (2020–2026) fatigue-aware and robust optimization literature (**C5**).

2 Related Work

2.1 Review Scheduling

Spaced repetition literature has progressed from heuristic interval multipliers [4] to stochastic optimal control [5], queueing models with explicit review budgets [6], discrete optimization over memory states [7], and stochastic shortest path algorithms [8, 9]. Skill-acquisition models such as DAS3H [10] and Pavlik & Anderson’s model [11] optimize practice schedules for complex concepts. Modern platforms utilize algorithms such as FSRs in Anki [12] or LLM-enhanced concept modeling in LECTOR [13]. These approaches treat time as an unconstrained budget and omit daily cognitive fatigue dynamics, hard task deadlines, and precedence constraints.

2.2 Fatigue-Aware Scheduling

Worker fatigue modeling originated in dual-resource constrained shop scheduling [14] and human learning-forgetting dynamics [15]. Recent extensions incorporate fatigue into flexible job shops using multi-objective metaheuristics [16], cobotic order picking [17], man-machine teaming overload control [18], and stochastic mission planning [19]. The closest robust variant combines proactive project network scheduling with fatigue minimization using NSGA-II [20]. Constraint programming approaches have addressed workload caps in preemptive job shops [21]. Machine learning frameworks like FALCON [22] model cumulative human workload as a constrained MDP state; however, FALCON assumes uniform fatigue per task, neglects intra-session recovery, and operates solely in simulation. Empirical studies in construction ergonomics identify near-linear strain accumulation alongside non-linear recovery curves [23, 24], emphasizing parameter identifiability challenges in real-world environments and supporting our use of hyper-parameter sensitivity analyses. Furthermore, mixed evidence regarding overnight memory consolidation [25] supports treating sleep recovery as a configurable hyper-parameter.

2.3 Robust and Chance-Constrained Scheduling

Resource-constrained project scheduling (RCPSP) [26, 27] and proactive-reactive frameworks [28] form the foundation for schedule buffering and rolling-horizon execution. Probabilistic constraints are solved via sample average approximation (SAA) [29]. Caserta [30] decouples uncertainty quantification from optimization by converting duration distributions into buffered operational windows. Conformal prediction techniques provide finite-sample coverage guarantees in chance-constrained data center operations [31] and contextual robust optimization [32]. Human task estimation exhibits known optimism bias [33, 34], which can be calibrated using few-shot prompting on frontier AI models [35].

2.4 Personal Scheduling and LLM + Solver Systems

Personal planning frameworks using CP-SAT incorporate soft constraints for time-of-day preferences, task grouping, and breaks [36]. Commercial platforms rely on rigid priority rules [37], while patented architectures outline cognitive-state-aware scheduling concepts [38]. Recent hybrid systems combine LLMs for informal task formalization with symbolic solvers for exact execution [39, 40, 41, 42, 43].

2.5 Theoretical Foundations

Dynamic neural state occupancy [44], predictive information growth [45], and free-energy minimization [46] motivate maintaining compact, adaptive user profiles.

2.6 The Research Gap

Existing fatigue-aware scheduling focuses on industrial shop floors using metaheuristics; personal planners rely on static preference scores; review schedulers omit capacity constraints; and LLM duration estimates are rarely propagated as uncertainty distributions into MIP capacity bounds. Our formulation bridges these domains by integrating circadian fatigue dynamics, chance constraints, and Bayesian memory gains into a unified model.

3 Diagnosis of the Axiom Baseline

An audit of Axiom’s open-source scheduling engine (`schedule.ts` and `engine.ts`) [1] reveals key structural deficiencies, summarized in Table 1.

Table 1: Diagnosis of existing Axiom scheduler implementations.

Observation	Consequence and Proposed Fix
1. Dual scheduling pipelines with conflicting cost logic	Incomparable outputs; preserved as distinct baselines B1 and B2.
2. Depleting budget ignores user time windows; energy curve ignores placed load	Neither pipeline combines fatigue and circadian dynamics; unified in Section 5.
3. Fitness function compares slot bandwidth (0.5–10) against total task load $ E_{\text{task}} $	Scale mismatch; redefined as per-slot load intensity $\lambda_i = \ell_i/p_i$.
4. Soft deadlines with unconstrained fallback overrides	Replaced with hard time windows, drop decision variables y_i , and precedence constraints.
5. Load formulas multiply priority, urgency, and effort parameters	Conflates task priority with mental effort; decoupled via M1 load function L .
6. Context-switch penalty counter capped at 1; candidate indices mismatched	Non-reproducible behavior; fixed with standard clock injection adapter.
7. Deterministic point estimates used exclusively	Zero robustness against M4 estimation errors; addressed by P-CC formulation.

4 Intuition: Capacity States, Chance Constraints, and Solvers

Capacity State. A user’s operational day is modeled as a battery whose maximum draw varies according to a circadian ceiling B_t . Working accumulates fatigue F_t , which degrades effective capacity. Placing three heavy cognitive tasks consecutively incurs compounding fatigue penalties, whereas distributing them across rest intervals maintains sustainable workload levels.

Chance Constraint. Rather than optimizing for single point estimates, the system evaluates S scenario realizations of task duration, cognitive difficulty, and circadian availability. The scheduler guarantees that the probability of daily cognitive overload remains below a specified risk threshold ϵ .

Solver Integration. CP-SAT provides provably optimal baseline schedules (**P-DET**). For larger instances, a metaheuristic simulated annealing algorithm navigates schedule candidate permutations, decoded via a serial schedule generation scheme.

5 Methodology

5.1 Time Discretization and Task Parameters

Time is partitioned into $D = 7$ days containing $H = 96$ fifteen-minute slots ($T = DH = 672$ slots). Availability $A_t \in \{0, 1\}$ marks valid operational slots excluding sleep, fixed commitments, and buffer windows. Sleep slots are identified by $z_t = 1$. Tasks $i \in \mathcal{I} = \mathcal{I}^w \cup \mathcal{I}^r$ (work vs. recreational) have planned duration p_i , recovery buffer β_i ($\bar{p}_i = p_i + \beta_i$), release time r_i , deadline d_i , difficulty δ_i , task type κ_i , priority π_i , subject k_i , group g_i , and sequence index σ_i . Feasible start slots are defined as:

$$\mathcal{S}_i = \{t : r_i \leq t, t + \bar{p}_i \leq \min(d_i, T), A_{t'} = 1 \forall t' \in [t, t + \bar{p}_i]\}. \quad (1)$$

Precedence constraints $(j, i) \in E$ enforce task execution order within groups.

5.2 Load Function

Module M1 provides task load $\ell_i = L(\delta_i, p_i, \kappa_i)$ and per-slot intensity $\lambda_i = \ell_i/p_i$ ($\lambda_i < 0$ for recreational items). Module M4 outputs predictive distributions $(\mu_p, \sigma_p, \mu_\delta, \sigma_\delta)$ formatted via inter-module JSON contracts.

5.3 Capacity Dynamics

Let B_t denote the circadian capacity ceiling and $F_t \geq 0$ denote accumulated fatigue. With $w_{i,t} \in \{0, 1\}$ indicating active execution of task i at slot t , active work load u_t^w , total load u_t , and work activity a_t^w are given by:

$$u_t^w = \sum_{i \in \mathcal{I}^w} \lambda_i w_{i,t}, \quad u_t = u_t^w + \sum_{i \in \mathcal{I}^r} \lambda_i w_{i,t}, \quad a_t^w = \sum_{i \in \mathcal{I}^w} w_{i,t}. \quad (2)$$

Fatigue updates recursively according to:

$$F_{t+1} = \max(0, F_t + u_t - \rho_t(1 - a_t^w)), \quad F_0 = F^0, \quad (3)$$

where $\rho_t = \rho^{\text{sleep}}$ when $z_t = 1$ and $\rho_t = \rho^{\text{idle}}$ otherwise. Feasibility requires:

$$u_t^w + \eta F_t \leq B_t \quad (\text{overload avoidance}). \quad (4)$$

5.4 Deterministic Formulation (P-DET)

Decision Variables. $s_{i,t} \in \{0, 1\}$ indicates task i starting at slot $t \in \mathcal{S}_i$; $y_i \in \{0, 1\}$ indicates task execution; start time $S_i = \sum_t t s_{i,t}$; active status $x_{i,t} = \sum_{t - \bar{p}_i < t' \leq t} s_{i,t'}$; work status $w_{i,t} = \sum_{t - p_i < t' \leq t} s_{i,t'}$; slacks $F_t, o_t, e_t, sw_t, \Delta_i \geq 0$.

Constraints.

$$\sum_{t \in \mathcal{S}_i} s_{i,t} = y_i \quad \forall i \in \mathcal{I}, \quad (5)$$

$$\sum_{i \in \mathcal{I}} x_{i,t} \leq 1 \quad \forall t \in \mathcal{T}, \quad (6)$$

$$y_i \leq y_j, \quad S_i \geq S_j + \bar{p}_j - T(1 - y_i) \quad \forall (j, i) \in E, \quad (7)$$

$$F_{t+1} \geq F_t + u_t - \rho_t(1 - a_t^w), \quad F_0 = F^0, \quad (8)$$

$$u_t^w + \eta F_t - B_t \leq o_t \quad \forall t \in \mathcal{T}, \quad (9)$$

$$F_t - F^{\text{soff}} \leq e_t, \quad F_t \leq F^{\text{max}}, \quad \sum_{t:d(t)=d} u_t^w \leq Q_d \quad \forall d, \quad (10)$$

$$sw_t \geq q_{k,t-1} + a_t - q_{k,t} - 1 \quad \forall k, t, \quad (11)$$

$$\Delta_i \geq \pm(S_i - S_i^{\text{prev}}) - T(1 - y_i) \quad \forall i, \quad (12)$$

where $a_t = \sum_i x_{i,t}$ and $q_{k,t} = \sum_{i:k_i=k} x_{i,t}$.

Objective Function.

$$\begin{aligned} \min J = & \sum_{i \in \mathcal{I}^w} c_i^{\text{drop}}(1 - y_i) + w_o \sum_t o_t + w_f \sum_t e_t + w_a \sum_{i,t} \mu_{i,t} s_{i,t} \\ & + w_s \sum_t sw_t + w_\Delta \sum_i \Delta_i - w_g \sum_{j \in \mathcal{I}^{\text{rev}}, t} g_j(t) s_{j,t} + w_\tau \sum_{j \in \mathcal{I}^{\text{rev}}, t} \tau_j(t) s_{j,t}, \end{aligned} \quad (13)$$

where $c_i^{\text{drop}} = \omega_{\text{drop}} \bar{\pi}_i$.

Lemma 1 (Tightness). *Replacing F_t in any feasible solution with the exact non-linear recursion yields a feasible solution with $J' \leq J$.*

Proof. By induction, exact fatigue values $\hat{F}_t \leq F_t$ since $\max(0, \cdot)$ is non-decreasing. Because F_t enters bounds (9)–(10) with non-negative coefficients, slacks do not increase. $\square$

Proposition 1 (Hardness). *Deciding task feasibility under P-DET is strongly NP-hard.*

Proof. Setting $\eta = 0$, $B_t = \infty$, and $E = \emptyset$ reduces P-DET to single-machine scheduling with release dates and deadlines $(1|r_i, d_i| -)$, which is strongly NP-hard [48]. $\square$

5.5 Uncertainty Handling: Chance-Constrained Extension (P-CC)

Task durations are padded using conservative quantiles $p_i = \lceil q_{1-\epsilon_p}(\tilde{p}_i)/15 \rceil$. We draw S scenario realizations $\omega = 1, \dots, S$ containing difficulties δ_i^ω , loads λ_i^ω , and capacity parameters $(B^\omega, \eta^\omega, \rho^\omega, F^{0,\omega})$ generated from Module M3 posterior samples. Overload risk per day d is bounded using SAA binary indicators $v_{d,\omega} \in \{0, 1\}$:

$$u_t^{w,\omega} + \eta^\omega F_t^\omega - B_t^\omega \leq M v_{d(t),\omega} \quad \forall t, \omega, \quad (14)$$

$$\sum_{\omega=1}^S v_{d,\omega} \leq \lfloor \epsilon S \rfloor \quad \forall d, \quad (15)$$

where $M = \max \lambda_i + \eta \max F^{\text{max}}$. In ablation A6, Gaussian quantiles are replaced with conformal prediction regions calibrated on historical completion logs [31, 32].

5.6 Memory Review Coupling

For flashcard review tasks $j \in \mathcal{I}^{\text{rev}}$, Module M3 provides marginal memory gain $g_j(t) = \mathbb{E}[\sigma(\psi_j^+(t)) - \sigma(\psi_j(t))]$, where ψ denotes logit-scale retention state [3]. Soft due slots $d_j^\theta = \min\{t : \hat{\rho}_j(t) < \theta\}$ incur lateness penalties $\tau_j(t) = c_j(t - d_j^\theta)^+$.

5.7 Solution Algorithms

Exact solutions are computed via Google CP-SAT [49]. Metaheuristic solutions use Simulated Annealing over priority lists decoded by a serial schedule generation scheme. Rolling-horizon execution updates schedules dynamically according to Algorithm 1. Solver explanations generate comparative cost breakdowns for alternative slot selections.

Algorithm 1 Rolling-Horizon Chance-Constrained Optimization (P-CC)

1. **Initialize:** Receive task set $\mathcal{I}$, M4 probability distributions, and M3 profile posterior samples.
 2. **Loop:** Upon trigger (task completion, new arrival, or periodic clock step):
 - (a) Freeze active/completed tasks; update initial fatigue F_0 from execution history.
 - (b) Sample S parameter scenarios $\omega \in \{1, \dots, S\}$ and compute padded durations p_i .
 - (c) Warm-start priority list from previous plan or EDD heuristic.
 - (d) **MetaData Search:** Iterate swap, insertion, and block-shift moves. Evaluate candidate cost J using single-pass forward propagation of capacity equations across S scenarios. Accept candidates via Metropolis criterion subject to SAA violation limit $\lfloor \epsilon S \rfloor$.
 - (e) **Refine:** Apply CP-SAT solver on local window for fine alignment.
 - (f) **Commit:** Commit next 24-hour schedule horizon; penalize displacement of unstarted future tasks via stability term Δ_i .
-

6 Evaluation Protocol

6.1 Baselines and Ablations

Evaluations compare P-DET and P-CC against six baselines listed in Table 2. Ablations comprise: **A1** (no fatigue tracking, $\eta = 0$), **A2** (two-component fast/slow fatigue), **A3** (alternate load functions E_{task} and `computeCL`), **A4** (omitting precedence and hard deadlines), **A5** (misspecified parameter sets η^*, ρ^*), and **A6** (conformal calibration).

Table 2: Evaluated baseline algorithms.

ID	Algorithm Method	Description
B0	Fixed-Interval Baseline	Load-blind standard calendar blocks.
B1	Axiom <code>runScheduler</code>	Primary baseline pipeline in Axiom repository [1].
B2	Axiom <code>runSchedulingAlgorithm</code>	Secondary fallback pipeline in Axiom repository.
B3	Earliest Due Date (EDD) + SGS	Priority list decoded via Schedule Generation Scheme.
B4	P-DET Exact Solver	Exact MIP solution via CP-SAT.
B5	P-DET Heuristic Solver	Simulated Annealing metaheuristic solver.
B6	P-CC Chance-Constrained	Stochastic sample-average approximation formulation.

6.2 Data and Simulation Harness

Synthetic benchmarks span $D = 7$ days, task counts $n \in \{10, 20, 40\}$, utilization ratios $\varrho = \sum p_i / \sum A_t \in \{0.4, 0.6, 0.8, 1.0\}$, and diverse chronotypes. Execution simulations incorporate optimistic duration bias, non-linear true strain accumulation, and random interruption events. Spaced repetition experiments utilize public MaiMemo logs [9] and Duolingo datasets [3], with FSRs [12] as a domain reference. An optional user pilot evaluates subjective workload using the MEL-TS scale [51].

6.3 Metrics and Hypotheses

Primary metrics include overload mass $\sum_t (u_t^w + \eta^* F_t^* - B_t^*)^+$, overload day frequency, deadline miss rate, and mean tardiness. Secondary metrics evaluate context switches, schedule stability, and retention

probability.

Hypotheses. **H1:** B4 reduces overload mass and deadline misses compared to B1–B3 under high utilization ($\rho \geq 0.6$). **H2:** The performance gap between B4 and greedy baselines widens as utilization increases. **H3:** P-CC (B6) maintains lower realized overload than P-DET under estimation noise $\varsigma > 0$. **H4:** Metaheuristic B5 achieves results within 5% of exact optimal solution B4 with orders-of-magnitude faster execution.

Table 3: Paired experimental comparison structure.

Comparison Pair	Δ Overload Mass	Δ Miss Rate	Holm-Adjusted p
B4 vs. B1 (Axiom Default)	Benchmark Pending	Benchmark Pending	–
B4 vs. B3 (EDD Baseline)	Benchmark Pending	Benchmark Pending	–
B6 vs. B4 (Noise $\varsigma > 0$)	Benchmark Pending	Benchmark Pending	–
B5 vs. B4 (Optimality Gap)	Benchmark Pending	Benchmark Pending	–

7 System Interfaces and Execution Plan

Inter-module data contracts are detailed in Table 4. The eight-week implementation plan covers: (W1) harness development; (W2) CP-SAT model implementation; (W3) capacity sensitivity experiments; (W4) chance-constrained integration; (W5) metaheuristic tuning; (W6) retention coupling; (W7) benchmark execution; and (W8) final write-up.

Table 4: Data contracts and interface map across modules.

Module	Provided Parameters	Integration Target in Engine
M1	Load function $L(\delta, p, \kappa)$, load intensity λ_i	Replaces static E_{task} heuristic.
M4	Difficulty $\tilde{\delta}_i$, duration $\tilde{p}_i$, estimation variance	Replaces static task difficulty fields.
M3	Availability A_t , capacity (B_t, η, ρ_t) , gain $g_j(t)$	Replaces fixed user commitment bounds.
Tasks	Precedence pairs $(j, i) \in E$, release times r_i	Maps to parent task relation IDs.

8 Discussion and Limitations

Evaluating model-based schedulers using simulated environments carries risk of structural bias. We mitigate this by employing mismatched data generators and evaluating all metrics under true latent parameters. Human fatigue parameters are unobservable in field settings, justifying our hyper-parameter sensitivity approach. Current models assume atomic tasks; extending the framework to preemptive splittable tasks represents future work [21]. Memory retention coupling depends on M3 parameter convergence, which is evaluated independently [3].

9 Conclusion

We presented a cognitive-load-aware task scheduling framework combining circadian fatigue dynamics, chance-constrained optimization, and memory retention gains into an exactly solvable mathematical model. The evaluation protocol tests solver robustness against misspecified execution models and established baselines.

References

- [1] Axiom Development Team. Axiom Learning Planner Engine (`schedule.ts`, `engine.ts`). GitHub Repository, 2026. <https://github.com/PMS61/Axiom>.

- [2] J. Sweller. Cognitive load during problem solving: Effects on learning. *Cognitive Science*, 12(2):257–285, 1988.
- [3] V. D. Pagnis, A. Verma, P. Sankhe, and V. Khandelwal. A hierarchical Bayesian state-space model with Pólya–Gamma augmented Gibbs sampling and dynamic Monte Carlo scheduling for cognitive-load-aware task planning. *FYP Update Report 2*, VJTI, 2026.
- [4] B. Settles and B. Meeder. A trainable spaced repetition model for language learning. In *Proc. ACL*, pages 1848–1858, 2016.
- [5] B. Tabibian et al. Enhancing human learning via spaced repetition optimization. *PNAS*, 116(10):3988–3993, 2019.
- [6] S. Reddy, I. Labutov, S. Banerjee, and T. Joachims. Unbounded human learning: Optimal scheduling for spaced repetition. In *Proc. ACM SIGKDD*, pages 1815–1824, 2016.
- [7] A. Hunziker et al. Teaching multiple concepts to a forgetful learner. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [8] J. Ye, J. Su, and Y. Cao. A stochastic shortest path algorithm for optimizing spaced repetition scheduling. In *Proc. ACM SIGKDD*, pages 4381–4390, 2022.
- [9] J. Su, J. Ye, L. Nie, Y. Cao, and Y. Chen. Optimizing spaced repetition schedule by capturing the dynamics of memory. *IEEE TKDE*, 35(10):9982–9996, 2023.
- [10] B. Choffin et al. DAS3H: Modeling student learning and forgetting for optimally scheduling distributed practice of skills. In *Proc. EDM*, 2019.
- [11] P. I. Pavlik and J. R. Anderson. Using a model to compute the optimal schedule of practice. *J. Exp. Psychol. Appl.*, 14(2):101–117, 2008.
- [12] Anki Documentation. Free Spaced Repetition Scheduler (FSRS) Integration. *Anki Manual*, 2026. [https://en.wikipedia.org/wiki/Anki_\(software\)](https://en.wikipedia.org/wiki/Anki_(software)).
- [13] J. Zhao. LECTOR: LLM-enhanced concept-based test-oriented repetition for adaptive spaced learning. *arXiv preprint arXiv:2508.03275*, 2025.
- [14] M. Y. Jaber and W. P. Neumann. Modelling worker fatigue and recovery in dual-resource constrained systems. *Comput. Ind. Eng.*, 59(1):75–84, 2010.
- [15] M. Y. Jaber, Z. S. Givi, and W. P. Neumann. Incorporating human fatigue and recovery into the learning-forgetting process. *Appl. Math. Model.*, 37(12-13):7287–7299, 2013.
- [16] W. Tan, X. Yuan, J. Wang, and X. Zhang. A fatigue-conscious dual resource constrained flexible job shop scheduling problem by enhanced NSGA-II. *Comput. Ind. Eng.*, 160:107557, 2021.
- [17] B. L. Vermin, D. A. Abbink, and F. Schulte. Bi-objective job-shop scheduling considering human fatigue in cobotic order picking. *Procedia Comput. Sci.*, 232:635–644, 2024.
- [18] H. Li, Y. Zhang, and X. Chen. Man-machine teaming: AI’s overload management and task allocation. In *Proc. ICCAS*, SCITEPRESS, pages 95–103, 2024.
- [19] S. H. Kim and D. H. Moon. Optimising stochastic task allocation and scheduling plans for mission workers subject to learning-forgetting, fatigue-recovery, and stress-recovery effects. *Int. J. Prod. Res.*, 61(18):6210–6228, 2023.
- [20] M. S. Kramer, P. A. Santos, and R. F. Silva. Robust optimization of multi-skill resource-constrained project networks considering staff fatigue under uncertainty. *Comput. Oper. Res.*, 162:106000, 2026.

- [21] T. Terrien. An iterative constraint programming approach to integrate maximum workload constraints in preemptive jobshop scheduling. In *Proc. CP 2026*, LIPIcs, arXiv:2605.20387, 2026.
- [22] Z. Zhang, C. C. Nguyen, D. Rosewarne, K. Wells, and G. Carneiro. Fatigue-aware learning to defer via constrained optimisation. *arXiv preprint arXiv:2604.00904*, 2026.
- [23] Y. Wang, B. Xiao, and S. T. Mueller. Understanding and modeling perceived cognitive and physical strain dynamics for planning-oriented human-robot collaboration in prefabricated construction. *arXiv preprint arXiv:2606.15494*, 2026.
- [24] Y. Wang, B. Xiao, and S. T. Mueller. Capturing human-centric adaptation in human-robot collaboration for prefabricated construction: Empirical modeling of mental and physical effort dynamics. In *Proc. ISARC*, 2026.
- [25] A. A. V. Guttesen, M. O. Harrington, M. G. Gaskell, and S. A. Cairney. Does overnight memory consolidation support next-day learning? *Cognition*, 264:106241, 2025.
- [26] E. Demeulemeester and W. Herroelen. *Project Scheduling: A Research Handbook*. Springer, 2002.
- [27] E. Demeulemeester and W. Herroelen. Robust project scheduling. *Found. Trends Technol. Inf. Oper. Manage.*, 3(3-4):201–376, 2011.
- [28] W. Herroelen and R. Leus. Robust and reactive project scheduling: A review and classification of procedures. *Int. J. Prod. Res.*, 42(8):1599–1620, 2004.
- [29] J. Luedtke and S. Ahmed. A sample approximation approach for optimization with probabilistic constraints. *SIAM J. Optim.*, 19(2):672–699, 2008.
- [30] M. Caserta. A robust chance constrained approach to surgery scheduling. *arXiv preprint arXiv:2608.03931*, 2026.
- [31] Y. Yang et al. A conformal prediction-based chance-constrained programming approach for 24/7 carbon-free data center operation scheduling. *arXiv preprint arXiv:2510.04053*, 2025.
- [32] Y. Patel, S. Rayan, and A. Tewari. Conformal contextual robust optimization. In *Proc. AISTATS*, PMLR 238:310–318, 2024.
- [33] R. Buehler, D. Griffin, and M. Ross. Exploring the “planning fallacy”: Why people underestimate their task completion times. *J. Pers. Soc. Psychol.*, 67(3):366–381, 1994.
- [34] S. R. Miller, J. A. Lee, and C. K. Davis. Time-on-task estimation for tasks lasting hours spread over multiple days. *PMC Journal Reference*, PMC12445496, 2025.
- [35] D. Gould et al. Think Fast: Estimating No-CoT task-completion time horizons of frontier AI models. *arXiv preprint arXiv:2606.07157*, 2026.
- [36] S. Volchkov. Optimizing personal schedules with CP-SAT and soft constraints. In *Creativity in Intelligent Technologies and Data Science*, pages 549–564. Springer, 2026.
- [37] Reclaim.ai. Automated Calendar Schedulers and Motion Alternatives Analysis. Vendor Technical Report, 2026. <https://reclaim.ai/blog/motion-alternatives>.
- [38] A. B. Smith et al. Cognitive scheduling platform and task planning engine. US Patent 11,093,904, 2021.
- [39] R. Amonkar et al. A reality check of language models as formalizers on constraint satisfaction problems. *arXiv preprint arXiv:2505.13252*, 2025.
- [40] Y. Li, N. Kamra, R. Desai, and A. Halevy. Human-centered planning. *arXiv preprint arXiv:2311.04403*, 2023.

- [41] S. L. Vasileiou and W. Yeoh. TRACE-CS: Explainable course scheduling using LLMs and logic. In *Proc. AAAI*, 39(28):1210–1218, 2025.
- [42] X. Liu et al. A preference-driven LLM-solver system for travel planning. In *Proc. ACL*, pages 4500–4512, 2025.
- [43] C. Zou, Y. Yao, S. She, N. Goodman, and R. D. Hawkins. CalBench: Evaluating coordination-privacy trade-offs in multi-agent LLMs. *arXiv preprint arXiv:2605.09823*, 2026.
- [44] D. Vidaurre, S. M. Smith, and M. W. Woolrich. Brain network dynamics are hierarchically organized in time. *PNAS*, 114(48):12827–12832, 2017.
- [45] W. Bialek, I. Nemenman, and N. Tishby. Predictability, complexity, and learning. *Neural Comput.*, 13(11):2409–2463, 2001.
- [46] K. Friston. The free-energy principle: A unified brain theory? *Nat. Rev. Neurosci.*, 11(2):127–138, 2010.
- [47] A. A. Borbély. A two process model of sleep regulation. *Hum. Neurobiol.*, 1(3):195–204, 1982.
- [48] M. R. Garey and D. S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, 1979.
- [49] L. Perron and F. Didier. CP-SAT Solver Technical Reference. Google OR-Tools Documentation, 2025.
- [50] S. G. Hart and L. E. Staveland. Development of NASA-TLX (Task Load Index): Results of empirical and theoretical research. *Adv. Psychol.*, 52:139–183, 1988.
- [51] T. Endres, L. Bender, S. Sepp, et al. Developing the Mental Effort and Load-Translingual Scale (MEL-TS) as a foundation for translingual research in self-regulated learning. *Educ. Psychol. Rev.*, 37(1):5, 2025.